Task 5 – Connect Local Repository to GitHub

Git & GitHub Fundamentals (Industry Workflow) | Estimated Time: 1 Hour

---

Step 1 – Create a GitHub Account

If you don't have a GitHub account yet, head over to **https://github.com** and sign up — it's free. One small but important thing: use the same email address you set up in Task 2 when you ran `git config --global user.email`. GitHub uses that email to recognise your commits and connect them to your profile.

> *Already have an account? Just sign in and jump to Step 2.*

Step 2 – Create the Remote Repository on GitHub

Think of this as creating an empty 'cloud copy' of your project. Once it exists on GitHub, you'll link your local folder to it.

Here's how to create it:

- Click the **+** icon in the top-right corner and choose **New repository**.
- Name it `DevOps-Day4` — exactly as shown.
- Choose Public or Private, whichever you prefer.
- **Very important:** do NOT tick the box to add a README, .gitignore, or licence. Your local repo already has files, and adding these here will cause a conflict when you try to push.
- Hit **Create repository**.

> *GitHub will now show you a quick-setup page. Grab the HTTPS URL at the top — it looks something like* `https://github.com/your-username/DevOps-Day4.git`*. You'll need it in the next step.*

Step 3 – Connect Your Local Repo to GitHub

Open your terminal, make sure you're inside the `DevOps-Git` folder, and run these three commands one by one. Paste your actual GitHub URL where you see `<repository-url>`.

```
git remote add origin <repository-url>
```

This tells your local Git: 'Hey, there's a remote repo at this URL — call it origin.' The name 'origin' is just a nickname — it's the standard everyone uses for the main remote.

```
git branch -M main
```

This renames your current branch to 'main'. Older versions of Git used to call it 'master' by default, but GitHub switched to 'main' a few years ago. This command just makes sure both sides match so there's no confusion when you push.

```
git push -u origin main
```

This is the moment your code goes to GitHub! All your local commits get uploaded to the remote repo. The `-u` part is a one-time setup — it remembers that 'origin main' is where you want to push. After this, you can just type `git push` and it'll know what to do.

> Heads up: GitHub will ask for your username and password. But it won't accept your actual password anymore — you need a Personal Access Token (PAT) instead. To get one, go to GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token. Give it repo access and copy it somewhere safe. Paste it as the password when Git asks.

Step 4 – Verify It Worked

Open your browser and visit your repo: `https://github.com/your-username/DevOps-Day4`. If everything went well, you should see your files sitting right there — README.md, notes.txt, commands.txt — and your commit history on the right side.

Want to double-check from the terminal too? Run this:

```
git remote -v
```

It'll print out the fetch and push URLs for 'origin'. If you see your GitHub link there, you're all connected and good to go.

What These Commands Actually Mean

Here's a plain-English breakdown of the terms you just worked with:

- **Remote** — A copy of your repo that lives on another machine — in this case, GitHub's servers. Your laptop has the local copy; GitHub has the remote one.
- **origin** — Just a nickname for your GitHub repo URL. You could call it anything, but everyone uses 'origin' — so stick with it.
- **git remote add** — Saves the GitHub URL into your local repo so Git knows where to push and pull from.
- **git branch -M main** — Renames your branch to 'main' to match what GitHub expects by default.
- **git push -u origin main** — Sends your commits to GitHub for the first time and sets up the default so future pushes are just 'git push'.
- **PAT (Personal Access Token)** — GitHub's secure alternative to passwords. You generate it once and use it wherever Git asks for a password over HTTPS.
- **git remote -v** — A quick way to check which remote URL your local repo is connected to.